On Pearl Blockchain's Proof of Useful Work: Systematization and Flexibility

Youlong Ding[†]

Abstract

We revisit the proof-of-useful-work (PoUW) scheme in Pearl blockchain's white paper (Pearl Research Team), giving a systematic sensitivity characterization as an alternative to its Jackpot policy, along with efficient algorithm for verifying the sensitivity condition.

Beyond its independent interest, our characterization yields a more flexible PoUW scheme that eenables miner autonomy: each miner can independently choose its noise magnitude to tailor the trade-off between computational accuracy and mining efficiency to its own needs.

[†]The Hebrew University of Jerusalem, Israel. Email: youlong.ding@mail.huji.ac.il.

Contents

1 Introduction **3**

2 Preliminaries **4**

3 Systematic Characterization: Sensitivity Condition **5**

4 Efficient Verification of Sensitivity Condition **5**
4.1 Inverting a monotone map 5
4.2 Backward propagation on the evaluation tree 5
4.3 Verification algorithm and analysis 6

5 POUW Scheme Made Flexible **7**

1 Introduction

We assume that the reader is familiar with Pearl blockchain's proof-of-useful-work (PoUW) protocol, as described in its white paper [Pea26] (also see [KW25]).

In this paper, we focus on the *Jackpot policy* in [Pea26, Section 5], which aims to prevent miners from exploiting adversarially chosen inputs to perform the prescribed matrix multiplication substantially faster than a generic computation.

At a high level, the Jackpot policy consists of four checks:

1. *Entry liveness.* Limits the fraction of input entries whose magnitudes overwhelm the injected noise, making their quantized values largely predictable.

2. *Noise floor.* Requires every input row to contain a sufficient amount of noise in quantized units.

3. *Tamed products.* Limits the fraction of output entries whose accumulation magnitudes are unusually large relative to the noise.

4. *Unpredictable summands.* Requires most summands to have sufficient variability relative to the accumulator's numerical precision.

While these checks address plausible attack vectors, we identify two limitations:

- First, the policy is rather *ad hoc*: it consists of several separate numerical conditions with carefully chosen thresholds, rather than a unified principle. This makes the policy difficult to understand and its completeness difficult to assess.

- Second, the policy prescribes a fixed relative noise magnitude, leaving miners little freedom to tailor the noise to their individual needs. In particular, different miners may prefer different trade-offs between computational accuracy and mining efficiency.

Our contributions. We propose a simple and systematic *sensitivity condition* as an alternative way to characterize the effect of injected noise. Informally, the condition requires that removing any single entry of either noise matrix, while keeping all other entries unchanged, must change the resulting matrix product. We formalize this condition over general finite types, allowing non-associative arithmetic such as floating-point accumulation.

We then give an efficient algorithm for verifying the sensitivity condition. For locally monotone operations, our algorithm runs in $O(n^3(1 + \log |\mathcal{S}|))$ time, nearly matching the cost of a single matrix multiplication. The key idea is to propagate intervals backward through the evaluation tree, allowing the effect of every noise entry to be tested without recomputing the matrix product.

Beyond its independent theoretical interest, our characterization also enables a more *flexible* PoUW design through *miner autonomy*. Rather than imposing a uniform noise magnitude, miners can independently choose their noise levels according to their own priorities. Smaller noise improves computational accuracy but may reduce the likelihood of satisfying the sensitivity condition, whereas larger noise may improve this likelihood at the cost of greater computational error.

2 Preliminaries

Definition 2.1 (Chain)**.** A *chain* is a finite totally ordered set

$$\mathcal{S} = \{s_1, \ldots, s_q\}, \qquad s_1 < \cdots < s_q, \qquad q \geq 1.$$

For $L, U \in \mathcal{S}$, $L \leq U$, define its interval

$$[L, U] \coloneqq \{s \in \mathcal{S} : L \leq s \leq U\}.$$

Definition 2.2 (Type)**.** A *type* is a triple $(\mathcal{S}, \oplus, \otimes)$, where $\mathcal{S}$ is a finite chain and $\oplus, \otimes : \mathcal{S} \times \mathcal{S} \to \mathcal{S}$ are binary operations satisfying the following properties:

1. *Commutativity:* For all $a, b \in \mathcal{S}$,

$$a \oplus b = b \oplus a, \qquad a \otimes b = b \otimes a.$$

2. *Zero element:* There exists a zero element $0 \in \mathcal{S}$ such that, for every $a \in \mathcal{S}$,

$$a \oplus 0 = a, \qquad a \otimes 0 = 0.$$

Example 2.3. Floating-point arithmetic, e.g., $(\mathsf{FP8}, +, \times)$, $(\mathsf{FP16}, +, \times)$ is a type.

Definition 2.4 (Tree evaluation)**.** Let $\oplus : \mathcal{S} \times \mathcal{S} \to \mathcal{S}$ be a binary operation on a chain $\mathcal{S}$. Let $\mathcal{T}$ be a binary tree with n leaves. Given $\mathbf{a} = (a_1, \ldots, a_n) \in \mathcal{S}^n$, we assign the value a_i to the i-th leaf. We recursively define the value $\mathsf{val}(v)$ of each node v:

$$\mathsf{val}(v) \coloneqq \begin{cases} a_i, & \text{if } v \text{ is the } i\text{-th leaf,} \\ \mathsf{val}(v_L) \oplus \mathsf{val}(v_R), & \text{if } v \text{ has left child } v_L \text{ and right child } v_R. \end{cases}$$

The *tree evaluation* is the value at the root:

$$\mathsf{TreeEval}_{\mathcal{T}}(\mathbf{a}, \oplus) \coloneqq \mathsf{val}(\mathrm{root}(\mathcal{T})).$$

Definition 2.5 (n-ary extension of a binary operation)**.** Let $(\mathcal{S}, \oplus, \otimes)$ be a type, and let $\mathcal{T}$ be a binary tree with n leaves. We define the n-ary extension of $\oplus$ induced by $\mathcal{T}$ as

$$\bigoplus\nolimits_{\mathcal{T}} \mathbf{a} \coloneqq \mathsf{TreeEval}_{\mathcal{T}}(\mathbf{a}, \oplus), \qquad \mathbf{a} \in \mathcal{S}^n.$$

Definition 2.6 (Local monotonicity)**.** Let $\mathcal{S}$ be a chain, $f : \mathcal{S}^n \to \mathcal{S}$. We say that f is *locally monotone* if, for every $i \in [n]$ and every fixed choice of $x_j \in \mathcal{S}$ for $j \neq i$, the unary map

$$f_i : \mathcal{S} \to \mathcal{S}, \qquad t \mapsto f(x_1, \ldots, x_{i-1}, t, x_{i+1}, \ldots, x_n)$$

is either nondecreasing or nonincreasing.

Definition 2.7 (Matrix multiplication)**.** Let $(\mathcal{S}, \oplus, \otimes)$ be a type, and let $\mathcal{T}$ be a binary tree with n leaves. For $\mathbf{A}, \mathbf{B} \in \mathcal{S}^{n \times n}$, we define their *matrix product* $\mathbf{A} \cdot_{\mathcal{T}} \mathbf{B} \in \mathcal{S}^{n \times n}$ by

$$(\mathbf{A} \cdot_{\mathcal{T}} \mathbf{B})_{i,j} \coloneqq \bigoplus\nolimits_{\mathcal{T}} \big(\mathbf{A}_{i,1} \otimes \mathbf{B}_{1,j}, \ldots, \mathbf{A}_{i,n} \otimes \mathbf{B}_{n,j}\big)$$

for all $i, j \in [n]$. When $\mathcal{T}$ is understood, we simply write $\mathbf{A} \cdot \mathbf{B}$.

3 Systematic Characterization: Sensitivity Condition

Definition 3.1 (Sensitivity condition)**.** Let $(\mathcal{S}, \oplus, \otimes)$ be a type. For $\mathbf{A}, \mathbf{B}, \mathbf{E}_1, \mathbf{E}_2 \in \mathcal{S}^{n\times n}$, let

$$\mathbf{C} \coloneqq (\mathbf{A} \oplus \mathbf{E}_1) \cdot (\mathbf{B} \oplus \mathbf{E}_2),$$

where $\oplus$ is naturally extended by applying it entrywise. We say that the *sensitivity condition* holds for $(\mathbf{A}, \mathbf{B}, \mathbf{E}_1, \mathbf{E}_2)$ with respect to $\mathcal{T}$ if, for every $(i, j) \in [n]^2$,

$$(\mathbf{A} + \mathbf{E}_1^{\setminus(i,j)}) \cdot (\mathbf{B} + \mathbf{E}_2) \neq \mathbf{C}, \qquad (\mathbf{A} + \mathbf{E}_1) \cdot (\mathbf{B} + \mathbf{E}_2^{\setminus(i,j)}) \neq \mathbf{C},$$

where $\mathbf{E}^{\setminus(i,j)}$ denote the matrix obtained from $\mathbf{E}$ by replacing its (i, j)-th entry with $0_{\mathcal{S}}$.

4 Efficient Verification of Sensitivity Condition

There are $2n^2$ possible noise-entry removals. Recomputing a full matrix product for each takes $O(n^5)$ time; even recomputing only the affected row or column takes $O(n^4)$ time. We show that local monotonicity of $\oplus$ allows all tests to be performed in $O(n^3(1 + \log|\mathcal{S}|))$ time.

Theorem 4.1 (Efficient verification)**.** *Let $(\mathcal{S}, \oplus, \otimes)$ be a type, and let $\mathcal{T}$ be a binary tree with n leaves. If $\oplus$ is locally monotone, then the sensitivity condition for any $\mathbf{A}, \mathbf{B}, \mathbf{E}_1, \mathbf{E}_2 \in \mathcal{S}^{n\times n}$ with respect to $\mathcal{T}$ can be verified in $O(n^3(1 + \log|\mathcal{S}|))$ time, assuming constant-time operations on $\mathcal{S}$ and indexed access to its ordered elements.*

4.1 Inverting a monotone map

Our basic tool is an interval inversion procedure that does not need to know the direction of monotonicity.

Lemma 4.2 (Interval inversion)**.** *Let $f : \mathcal{S} \to \mathcal{S}$ be either nondecreasing or nonincreasing, and let $I \subseteq \mathcal{S}$ be an interval. Suppose that $x_0 \in \mathcal{S}$ satisfies $f(x_0) \in I$. Then $f^{-1}(I)$ is an interval containing x_0, and its endpoints can be found using $O(1 + \log|\mathcal{S}|)$ evaluations of f.*

Proof. For $x \leq y \leq z$, monotonicity implies that $f(y)$ lies between $f(x)$ and $f(z)$. Thus $f(x), f(z) \in I$ implies $f(y) \in I$, so $f^{-1}(I)$ is an interval. Since $x_0 \in f^{-1}(I)$, the predicate $f(x) \in I$ is false and then true to the left of x_0, and true and then false to its right. Two binary searches on the ordered elements of $\mathcal{S}$ therefore find the endpoints, regardless of the monotonicity direction. □

4.2 Backward propagation on the evaluation tree

Write

$$\mathbf{X} \coloneqq \mathbf{A} \oplus \mathbf{E}_1, \qquad \mathbf{Y} \coloneqq \mathbf{B} \oplus \mathbf{E}_2, \qquad \mathbf{C} \coloneqq \mathbf{X} \cdot_{\mathcal{T}} \mathbf{Y},$$

where matrix addition is entrywise. Fix an output entry (i, j). Its evaluation tree $\mathcal{T}$ has leaf values

$$p_k \coloneqq \mathbf{X}_{i,k} \otimes \mathbf{Y}_{k,j}, \qquad k \in [n],$$

and root value $\mathbf{C}_{i,j}$. Evaluate the tree once and store $\mathsf{val}(v)$ for every node v.

For each node v, let $I_v \subseteq \mathcal{S}$ be the set of values that can replace $\mathsf{val}(v)$ without changing the root value, while all other parts of the computation remain fixed. In particular,

$$I_{\mathrm{root}(\mathcal{T})} = \{\mathbf{C}_{i,j}\}.$$

If v has left child u and right child w, then

$$\begin{aligned} I_u &= \{t \in \mathcal{S} : t \oplus \mathsf{val}(w) \in I_v\}, \\ I_w &= \{t \in \mathcal{S} : \mathsf{val}(u) \oplus t \in I_v\}. \end{aligned} \tag{1}$$

Because $\oplus$ is locally monotone, each set is the inverse image of an interval under a monotone unary map. Moreover, $\mathsf{val}(u) \in I_u$ and $\mathsf{val}(w) \in I_w$. By Lemma 4.2, both child intervals can therefore be computed in $O(1 + \log |\mathcal{S}|)$ time. Propagating these intervals from the root to the leaves gives, for each $k \in [n]$, the interval I_k of leaf values that preserve $\mathbf{C}_{i,j}$.

Now consider removing one noise entry. Replacing $(\mathbf{E}_1)_{i,k}$ by 0 changes only the k-th leaf of this output computation, whose new value is

$$p^{(1)}_{i,j,k} \coloneqq \mathbf{A}_{i,k} \otimes \mathbf{Y}_{k,j}.$$

Similarly, replacing $(\mathbf{E}_2)_{k,j}$ by 0 gives

$$p^{(2)}_{i,j,k} \coloneqq \mathbf{X}_{i,k} \otimes \mathbf{B}_{k,j}.$$

Consequently, the output entry (i, j) changes under the first or second removal exactly when, respectively,

$$p^{(1)}_{i,j,k} \notin I_k \qquad \text{or} \qquad p^{(2)}_{i,j,k} \notin I_k. \tag{2}$$

No monotonicity assumption on $\otimes$ is needed: each candidate leaf value is computed directly.

4.3 Verification algorithm and analysis

Initialize two $n \times n$ arrays of counters, $d^{(1)}$ and $d^{(2)}$, to zero. For each $(i, j) \in [n]^2$, evaluate the corresponding tree and propagate the intervals as in (1). For every leaf k, increment $d^{(1)}_{i,k}$ if $p^{(1)}_{i,j,k} \notin I_k$, and increment $d^{(2)}_{k,j}$ if $p^{(2)}_{i,j,k} \notin I_k$. The algorithm accepts if and only if every counter is positive.

Proof of Theorem 4.1. The intervals computed by backward propagation have exactly the stated meaning. This holds at the root, and (1) preserves it when passing from a parent to either child while fixing the other child's value. Thus (2) correctly determines whether removing a noise entry changes the output (i, j).

Removing $(\mathbf{E}_1)_{i,k}$ can affect only row i of the matrix product. Hence $d^{(1)}_{i,k} > 0$ exactly when this removal changes the output matrix. Likewise, removing $(\mathbf{E}_2)_{k,j}$ can affect only column j, and it changes the output matrix exactly when $d^{(2)}_{k,j} > 0$. Therefore all counters are positive precisely when the sensitivity condition holds.

Each output entry requires a forward evaluation of $O(n)$ nodes, a backward traversal with $O(1+\log |\mathcal{S}|)$ work per node, and $O(n)$ leaf tests. Over all n^2 output entries, this takes $O(n^3(1+\log |\mathcal{S}|))$ time. The matrices and counters use $O(n^2)$ space, and the values and intervals for one tree use $O(n)$ additional space. □

5 POUW Scheme Made Flexible

Beyond its independent interest, our systematic characterization enables a desirable feature of the resulting POUW scheme: miner autonomy. Namely, rather than imposing a uniform noise magnitude rule as in [Pea26], the resulting POUW scheme allows each miner to choose its own noise level, tailoring the trade-off between computational accuracy and mining efficiency to its individual needs:

1. *Smaller noise.* A miner prioritizing accuracy may choose smaller noise, reducing the deviation from the original matrix product, but potentially lowering the probability that the sensitivity condition holds.

2. *Larger noise.* A miner prioritizing mining efficiency may choose larger noise, potentially increasing the probability of satisfying the sensitivity condition at the cost of greater computational error.

Acknowledgements

Ding was supported in part by a grant from the Israel Science Foundation (ISF Grant No. 1774/20), and by the European Union (ERC, SCALE,101162665). Views and opinions expressed are however those of the author(s) only and do not necessarily reflect those of the European Union or the European Research Council. Neither the European Union nor the granting authority can be held responsible for them.

References

[KW25] Ilan Komargodski and Omri Weinstein. Proofs of useful work from arbitrary matrix multiplication. *arXiv preprint arXiv:2504.09971*, 2025.

[Pea26] Pearl Research Team. Pearl floating point scheme specification. Technical report, Pearl Research Labs, September 2026.